

FactoryOS AI Architecture

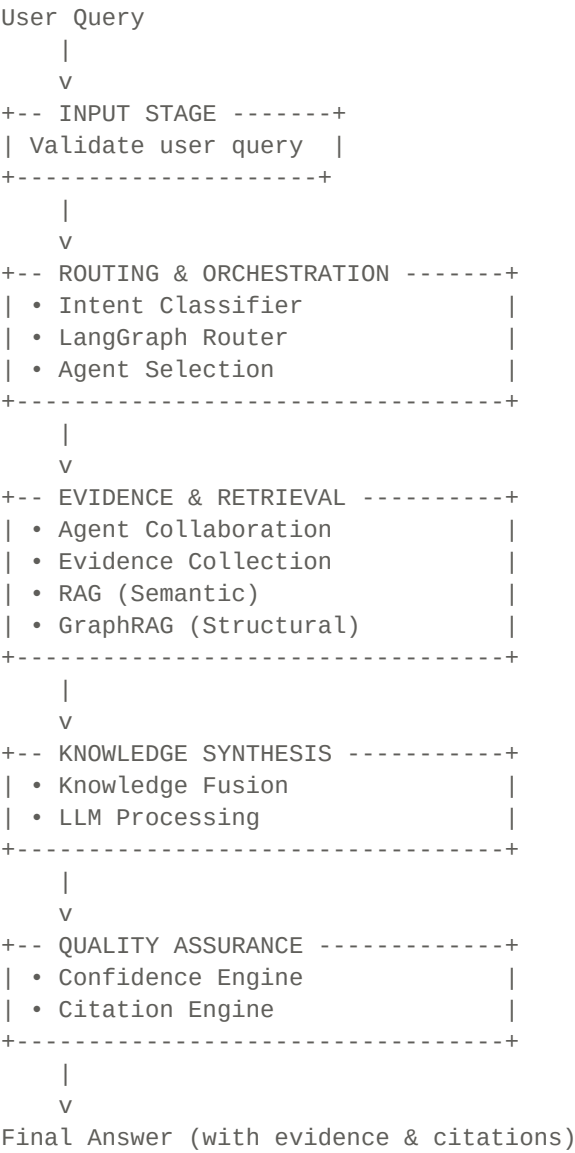
Multi-Stage Agentic Pipeline for Enterprise Knowledge Systems

Architecture Type	Sequential agentic pipeline
Primary Function	Intelligent query processing with evidence-based responses
Core Innovation	Parallel retrieval + confidence scoring + citation tracking
Generated	July 21, 2026

Executive Summary

FactoryOS AI implements a sophisticated multi-stage reasoning pipeline that transforms user queries into high-confidence, cited responses. The architecture separates concerns across routing, evidence gathering, knowledge fusion, and quality assurance stages. This design enables scalable deployment in enterprise contexts where explainability and accuracy are non-negotiable.

Architecture Pipeline



Key Design Principles

Separation of Concerns: Each stage handles a single responsibility—understanding, routing, retrieval, synthesis, and validation are decoupled.

Parallel Processing: RAG and GraphRAG execute simultaneously to capture both semantic and structural knowledge.

Evidence-First Design: All claims are traceable to source data; confidence and citations are mandatory outputs.

Scalable Agent Orchestration: The LangGraph Router dynamically selects and coordinates agents based on query intent.

Stage-by-Stage Breakdown

1. INPUT: User Query

Entry point. Validates that the query is well-formed and sufficient context is provided for routing.

2. Intent Classifier

Semantic parsing layer. Classifies the user's intent (query, analysis request, prediction, anomaly detection, etc.) to inform downstream agent selection.

3. LangGraph Router

Orchestration engine. Routes the classified intent to a subgraph of specialized agents designed to handle that intent class.

4. Agent Selection

Dynamic activation. Based on intent and available resources, selects which agents (domain experts, data retrieval tools, simulators) will be invoked.

5. Agent Collaboration

Parallel execution. Multiple agents execute concurrently on different aspects of the problem, sharing intermediate results via a shared context store.

6. Evidence Collection

Data gathering. Agents collect supporting evidence, run calculations, and surface relevant data points that will back the final response.

7. RAG (Semantic Retrieval)

Vector-based search. Retrieves documents, examples, and similar past queries from a dense vector index using semantic similarity.

8. GraphRAG (Structural Retrieval)

Knowledge graph queries. Executes structured queries over a knowledge graph to surface relationships, entity properties, and hierarchical data.

9. Knowledge Fusion

Synthesis layer. Merges results from RAG, GraphRAG, and agent-computed evidence into a unified context for the LLM.

10. LLM Processing

Response generation. The large language model generates a coherent, fluent response grounded in the fused knowledge base.

11. Confidence Engine

Quality scoring. Computes confidence scores based on evidence agreement, retrieval similarity, and agent consensus.

12. Citation Engine

Traceability. Attaches citations to each claim in the response, linking back to specific evidence and sources.

13. Final Answer

Output delivery. Returns a response with integrated confidence scores and full citation trails for audit and explainability.

Use Cases & Implementation Considerations

Ideal Use Cases

1. Industrial asset intelligence (predictive maintenance, anomaly detection, supply chain risk)
2. Enterprise knowledge systems requiring explainability and regulatory compliance
3. Multi-domain problem solving (combining financial, technical, and operational perspectives)
4. Systems requiring cited evidence for audit trails and decision justification
5. Scenarios with heterogeneous data sources (databases, APIs, knowledge graphs, documents)

Implementation Priorities

Agent Design: Each agent should be highly specialized and return structured evidence (not free-form text) for better fusion quality.

Vector Index Quality: RAG effectiveness is directly tied to embedding quality and index freshness—invest in document chunking strategy.

Knowledge Graph Schema: GraphRAG requires a well-designed ontology; schema design is critical for query expressiveness.

Confidence Calibration: Train the Confidence Engine on a labeled validation set to avoid overconfidence or underconfidence.

Citation Tracking: Implement immutable evidence logs; each citation must be reversible to its source with timestamps.

Latency Optimization: Cache repeated queries, run RAG/GraphRAG in parallel, and stream LLM output for perceived responsiveness.